PHISHING WEBSITE CLASSIFICATION WITH DECISION TREE

DATASET

For this part, I used the Website Phishing Dataset from the UCI Machine Learning Repository.
The dataset contains information about websites and is used to classify whether a website is phishing, suspicious, or legitimate.

Dataset source: https://archive.ics.uci.edu/dataset/379/website+phishing

Dataset statistics:
By using **DataFrame.shape[<index>]** attribute:

- Target attribute: Result
- Number of instances: **1353**
- Number of attributes: **9**
- Class distribution:

Class	Meaning	Number of instances
-1	Phishing	702
1	Suspicious	548
0	Legitimate	103

Dataset format: ARFF file

The dataset is stored in an **ARFF file**. I loaded the file using the arff module from scipy.io.

After loading the file, I converted it into a pandas DataFrame because it makes the data easier to work with in Python.

The values were also converted into numeric values using pd.to_numeric() before using them in the Decision Tree model.

PROBLEM

The goal of this task is to build a Decision Tree classifier that can classify a website into one of the three classes:

- phishing
- suspicious
- legitimate

The model uses the 9 website attributes as input features and tries to predict the Result class.

This is a **classification problem** because the output is a class instead of a continuous number.

DATA PREPARATION

First, I separated the dataset into the input features X and the target variable y.

The first 9 columns were used as the features, while the last column, Result, was used as the target.

I then divided the dataset into:

- **70% training data**
- **30% testing data**

I also used stratify=y so that the class distribution in the training and testing sets stayed similar to the original dataset.

A fixed random_state was used so that the same train/test split could be produced again.

DECISION TREE APPROACH

I first created a basic Decision Tree classifier using:

DecisionTreeClassifier(criterion="entropy")

The model was trained using the training data and then tested using the testing data.

The baseline model achieved an accuracy of:

87.19%

After that, I used GridSearchCV to test different values for two Decision Tree hyperparameters: max_depth, min_samples_split

I used **5-fold cross-validation** and accuracy as the scoring method.

The purpose of GridSearchCV was to compare different hyperparameter combinations and choose the combination with the best average validation accuracy.

RESULT

The results were:

Model	**Test Accuracy**
Baseline Decision Tree	87.19%
Tuned Decision Tree	85.71%

The tuned model had a slightly lower test accuracy than the baseline model.

At first, I expected the tuned model to always have a higher accuracy. However, GridSearchCV chooses the best model based on the validation results from the training data. It does not use the final test set when selecting the model.

Because of this, the model with the best cross-validation result does not always have the highest accuracy on the test set.

The difference between the two models was around **1.48%**.

For this dataset and this train/test split, the baseline Decision Tree gave a slightly higher test accuracy.

FEATURE IMPORTANCE

I also checked the importance of each attribute using:

feature_importances_

Feature importance shows how much each feature helped the Decision Tree make its decisions.

A higher feature importance value means that the feature contributed more to reducing the impurity of the tree.

In my result, **SFH** had the highest feature importance value, so it was the most important attribute used by the Decision Tree.

The feature importance values were sorted from highest to lowest so that the important attributes were easier to compare.

DECISION TREE VISUALIZATION

Finally, I used tree.plot_tree() from scikit-learn to visualize the Decision Tree.

The visualization shows the features and conditions used at each node of the tree. It also shows how the data is divided until the model reaches a final prediction.

This helped me understand how the Decision Tree uses the website attributes to classify each website.

CONCLUSION

In this part, I used a Decision Tree to classify websites as phishing, suspicious, or legitimate.

The baseline model achieved **87.19% accuracy**, while the model selected using GridSearchCV achieved **85.71% accuracy** on the test set.

Even though the tuned model had a lower test accuracy, GridSearchCV was still useful because it allowed different hyperparameter combinations to be tested using cross-validation instead of choosing the values manually.

The feature importance result also showed which website attributes had the biggest effect on the Decision Tree's predictions.